The Journal of Open Source Software

rollama: An R package for using generative large language models through Ollama

Johannes B. Gruber [1¶] **and Maximilian Weber** [2]

1 University of Amsterdam **2** Goethe University Frankfurt ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 21 March 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

`rollama` is an R package that wraps the `Ollama` API, which allows you to run different Generative Large Language Models (GLLM)[1] locally. The package and learning material focus on making it easy to use `Ollama` for annotating textual or imagine data with open-source models as well as use these models for document embedding. But users can use or extend `rollama` to do essentially anything else that is possible through OpenAI's API, yet more private, reproducible and for free.

Statement of need

As researchers embrace the next revolution in computational social science, the arrival of GLLM, there is a critical need for open-source alternatives (Spirling, 2023). This need arises to avoid falling into a reproducibility trap or becoming overly dependent on services offered by for-profit companies.

After the release of ChatGPT, researchers started using OpenAI's API for annotating text with GPT models (e.g., Gilardi et al., 2023; He et al., 2023). However, this approach presents several shortcomings, including privacy and replication issues associated with relying on proprietary models over which researchers have no control whatsoever (Spirling, 2023; Weber & Reichardt, 2023).

Fortunately, since GLLMs were popularized by OpenAI's ChatGPT a little more than a year ago, a large and active alliance of open-source communities and technology companies has made considerable efforts to provide open models that rival, and sometimes surpass, proprietary ones (Alizadeh et al., 2023; Irugalbandara et al., 2024).

One method of utilizing open models involves downloading them from a platform known as Hugging Face and using them via Python scripts. However, there is now software available that facilitates access to these models in an environment, allowing users to simply specify the model or models they wish to use. This can be done locally on one's computer, and the software is called `Ollama`. The R package `rollama` provides a wrapper to use the `Ollama` API within R.

Background: `Ollama`

`Ollama` can be installed using dedicated installers for macOS and Windows, or through a bash installation script for Linux[2]. However, our preferred method is to utilize Docker – an open-source containerization tool. This approach enhances security and simplifies the processes

[1] Also referred to generative AI or Generative Pre-trained Transformer (GPT).
[2] https://ollama.com/download

of updating, rolling back, and completely removing `Ollama`. For convenience, we provide a Docker compose file to start a container running `Ollama` and Open WebUI – a browser interface strongly inspired by ChatGPT – in a GitHub Gist[3].

Usage

After `Ollama` is installed, the R-package `rollama` can be installed from CRAN (the Comprehensive R Archive Network):

```
install.packages("rollama")
```

or from GitHub using remotes:

```
# install.packages("remotes")
remotes::install_github("JBGruber/rollama")
```

After that, the user should check whether the `Ollama` API is up and running.

```
library(rollama)
ping_ollama()
```

After installation, the first step is to pull one of the models from ollama.com/library by using the model tag. By calling `pull_model()` without any arguments will download the default model.

```
# pull the default model
pull_model()
# pull a specified model by providing the model tag
pull_model("gemma:2b-instruct-q4_0")
```

Main functions

The core of the package are two functions: `query()` and `chat()`. The difference is that `chat()` saves the history of the conversation with an `Ollama` model, while `query()` treats every question as a new conversation.

To ask a single question, the `query()` function can be used.

```
query("why is the sky blue?")
```

For conversational interactions, the `chat()` function is used.

```
chat("why is the sky blue?")
chat("and how do you know that?")
```

Reproducible outcome

Within the `model_params` of the `query()` function, setting a seed is possible. When a seed is used, the temperature must be set to "0" to ensure consistent output for repeated prompts.

```
query("Why is the sky blue? Answer in one sentence.",
      model_params = list(
        seed = 42,
        temperature = 0
      ))
```

[3] https://gist.github.com/JBGruber/73f9f49f833c6171b8607b976abc0ddc

DRAFT

Examples

We present several examples to illustrate some functionalities. As mentioned previously, many tasks that can be performed through OpenAI's API can also be accomplished by using open models within `Ollama`. Moreover, these models can be controlled with a seed, ensuring reproducible results.

Annotating text

For annotating text data, various prompting strategies are available (see Weber & Reichardt, 2023 for a comprehensive overview). These strategies primarily differ in whether or how many examples are given (Zero-shot, One-shot, or Few-shot) and whether reasoning is involved (Chain-of-Thought).

When writing a prompt we can give the model content for the system part, user part and assistant part. The system message typically includes instructions or context that guides the interaction, setting the stage for how the user and the assistant should interact. For an annotation task we could write: "You assign texts into categories. Answer with just the correct category." The following example is a zero-shot approach only containing a system message and one user message. In practice, annotating a single text is rarely the goal. For guidance on batch annotations[4] and working with dataframes[5], refer to the package documentation.

```r
library(tibble)
library(purrr)
q <- tribble(
  ~role,    ~content,
  "system", "You assign texts into categories. Answer with just the correct category.",
  "user",   "text: the pizza tastes terrible\ncategories: positive, neutral, negative"
)
query(q, model_params = list(seed = 42, temperature = 0))
```

Using multimodal models

`Ollama` also supports multimodal models, which can interact with (but at the moment not create) images. A model designed for image handling, such as the llava model, needs to be pulled. Using `pull_model("llava")` will download the model.

```r
pull_model("llava")
query("Excitedly desscribe this logo", model = "llava",
      images = "https://ollama.com/public/ollama.png")
```

Obtaining text embeddings

`Ollama`, and hence rollama, can be utilized to generate text embeddings. In short, text embedding uses the knowledge of the meaning of words inferred from the context they are used in and saved in a large language model through its training to turn text into meaningful vectors of numbers (Kroon et al., 2023). This technique is a powerful preprocessing step for supervised machine learning and often increases the performance of a classification model substantially (Laurer et al., 2024). GLLMs can produce high-quality document embeddings, such as the current default model of `Ollama` llama2. However, to speed up the procedure, one can use specialized embedding models like nomic-embed-text[6] or all-minilm[7].

[4] https://jbgruber.github.io/rollama/articles/annotation.html#batch-annotation
[5] https://jbgruber.github.io/rollama/articles/annotation.html#another-example-using-a-dataframe
[6] https://ollama.com/library/nomic-embed-text
[7] https://ollama.com/library/all-minilm

DRAFT

```
pull_model(model = "nomic-embed-text")
embed_text(text = "It's a beautiful day", model = "nomic-embed-text")
```

Transformer based text embedding models were previously only available in Python, with the only pathway for R users to employ them being the Python interface provided by `reticulate` (Kalinowski et al., 2024) or packages built on top of it (e.g. Chan, 2023; Kjell et al., 2023). While these usually work well, non-technical R users regularly struggle with the requirement to set up the necessary Python environments. `Ollama`, and hence `rollama`, circumvents this step by packaging the whole application in an executable or Docker image. For a more detailed example of using embeddings for training supervised machine learning models and doing classification tasks, refer to the package documentation[8].

Learning material

We provide tutorials for the package at jbgruber.github.io/rollama and an initial overview is available as a YouTube video[9].

References

Alizadeh, M., Kubli, M., Samei, Z., Dehghani, S., Bermeo, J. D., Korobeynikova, M., & Gilardi, F. (2023). *Open-source large language models outperform crowd workers and approach ChatGPT in text-annotation tasks*. arXiv. https://doi.org/10.48550/arXiv.2307.02179

Chan, C. (2023). Grafzahl: Fine-tuning transformers for text data from within r. *Computational Communication Research*, *5*(1), 76–84. https://doi.org/10.5117/CCR2023.1.003.CHAN

Gilardi, F., Alizadeh, M., & Kubli, M. (2023). ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, *120*(30). https://doi.org/10.1073/pnas.2305016120

He, X., Lin, Z., Gong, Y., Jin, A.-L., Zhang, H., Lin, C., Jiao, J., Yiu, S. M., Duan, N., & Chen, W. (2023). *AnnoLLM: Making large language models to be better crowdsourced annotators*. arXiv. https://doi.org/10.48550/arXiv.2303.16854

Irugalbandara, C., Mahendra, A., Daynauth, R., Arachchige, T. K., Flautner, K., Tang, L., Kang, Y., & Mars, J. (2024). *A trade-off analysis of replacing proprietary LLMs with open source SLMs in production*. arXiv. https://doi.org/10.48550/arXiv.2312.14972

Kalinowski, T., Ushey, K., Allaire, J. J., Tang, Y., Eddelbuettel, D., Lewis, B., Keydana, S., Hafen, R., & Geelnard, M. (2024). *Reticulate: Interface to 'python'*. https://CRAN.R-project.org/package=reticulate

Kjell, O., Giorgi, S., & Schwartz, A. H. (2023). The text-package: An r-package for analyzing and visualizing human language using natural language processing and deep learning. *Psychological Methods*. https://doi.org/10.1037/met0000542

Kroon, A., Welbers, K., Trilling, D., & Atteveldt, W. van. (2023). Advancing automated content analysis for a new era of media effects research: The key role of transfer learning. *Communication Methods and Measures*, *0*(0), 1–21. https://doi.org/10.1080/19312458.2023.2261372

Laurer, M., Atteveldt, W. van, Casas, A., & Welbers, K. (2024). Less annotating, more classifying: Addressing the data scarcity issue of supervised machine learning with deep transfer learning and BERT-NLI. *Political Analysis*, *32*(1), 84–100. https://doi.org/10.1017/pan.2023.20

[8] https://jbgruber.github.io/rollama/articles/text-embedding.html
[9] https://youtu.be/N-k3RZqiSZY

Spirling, A. (2023). Why open-source generative AI models are an ethical way forward for science. *Nature*, *616*(7957), 413–413. https://doi.org/10.1038/d41586-023-01295-4

Weber, M., & Reichardt, M. (2023). *Evaluation is all you need. Prompting generative large language models for annotation tasks in the social sciences. A primer using open models.* arXiv. https://doi.org/10.48550/arXiv.2401.00284

DRAFT